Hydroponics Weather Advisor Toolkit

Prompt-Powered Kickstart: Building a Hydroponics Weather Advisor with Go

Author: Ashley Kinyanjui **Institution:** Moringa School — AI Essentials for Software Developers **Date:** 19th March 2026

---

1. Title & Objective

- **Project Title:** Hydroponics Weather Advisor Toolkit
- **Technology Chosen:** Go (Golang) + OpenWeatherMap REST API
- **Why I Chose It:** The Moringa School AI Essentials capstone required exploring a technology outside of Python and JavaScript, which pushed me to step beyond familiar territory. I chose Go (Golang) because it is a modern, statically typed, compiled language that is widely used in real-world backend systems. This made it both a relevant and practical choice — Go forced me to think differently about types, error handling, and program structure in a way that directly builds on the software development skills covered in the course.
- **End Goal:** Create a command-line tool for Nairobi farmers that fetches live temperature data and provides automated nutrient solution advice based on current weather conditions.

---

2. Quick Summary of the Technology

What is Go? Created by Google in 2009, Go is an open-source, compiled programming language built for speed, simplicity, and reliability. It is statically typed, meaning errors are caught before the program even runs.

Where is it used? Go powers major internet infrastructure including Docker (the world's most popular containerisation platform), Kubernetes (Google's cloud orchestration system), and Cloudflare (which handles traffic for millions of websites). It is widely used in backend APIs, DevOps tools, and cloud services.

One real-world example: Docker — used by over 20 million developers worldwide to package and ship software — is written entirely in Go. Its lightweight, fast-compiling nature makes it ideal for tools that need to start instantly and handle thousands of tasks simultaneously.

What is the OpenWeatherMap API? OpenWeatherMap is a web service that delivers live meteorological data from weather stations and satellites directly to our code in JSON format. It provides

temperature, humidity, wind speed, and more for any city in the world, and offers a free tier suitable for student and personal projects.

---

3. System Requirements

Requirement	Detail
Operating System	Windows 11
Code Editor	Visual Studio Code (VS Code)
Language	Go 1.26.1
Internet Connection	Required (for live API calls)
Packages	net/http, encoding/json (Go Standard Library — no extra installs needed)
API Key	Free OpenWeatherMap account

---

4. Installation & Setup Instructions

Step 1: Install Go Downloaded the Windows installer from go.dev/dl/ and ran it. Clicked through all default settings.

Step 2: Verify Setup Ran `go version` in the VS Code terminal to confirm the PATH was correctly configured.

Expected output:

```
go version go1.26.1 windows/amd64
```

Step 3: Install the Go Extension in VS Code Searched for "Go" by the Go Team at Google in the VS Code Extensions panel and installed it. This gives syntax highlighting and error detection.

Step 4: Initialize Project Created a project folder and ran:

```
go mod init hydroponics_advisor
```

Expected output:

```
go: creating new go.mod: module hydroponics_advisor
```

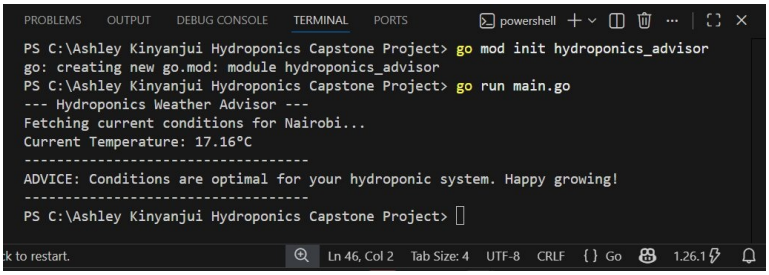

Figure 1: go mod init and go run main.go showing the program running successfully in the VS Code terminal.

Figure 1: Module initialization and first successful run of the Hydroponics Weather Advisor.

Step 5: Project Files in VS Code

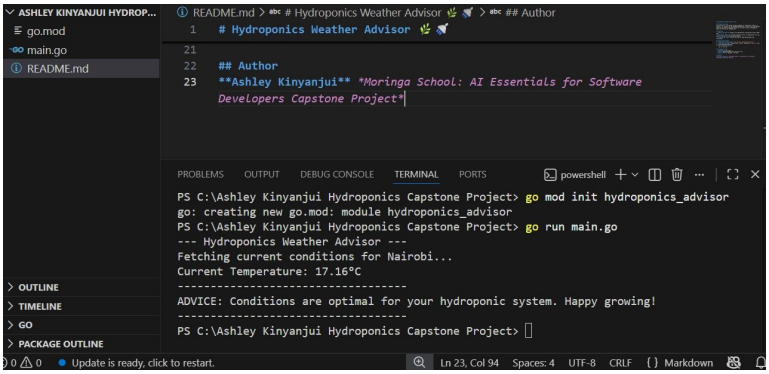

Figure 2: Project files (main.go, go.mod, README.md) visible in the VS Code Explorer panel alongside the terminal output.

Figure 2: The complete project structure in VS Code showing all three project files.

Step 6: API Access Generated a free API key at OpenWeatherMap.org by creating an account and navigating to the API Keys section of the dashboard. Note: new keys take up to 10 minutes to activate.

---

5. Minimal Working Example

This program requests live weather data for Nairobi and evaluates whether the current temperature requires action from the hydroponic farmer — specifically whether high heat requires higher oxygenation for plant roots, or whether cold temperatures may slow nutrient uptake.

Code (main.go):

```go
package main

import (
    "encoding/json"
    "fmt"
    "net/http"
)

// weatherData matches the JSON structure returned by OpenWeatherMap
type weatherData struct {
    Main struct {
        Temp     float64 `json:"temp"`
        Humidity float64 `json:"humidity"`
    } `json:"main"`
    Weather []struct {
        Description string `json:"description"`
    } `json:"weather"`
}

func main() {
    // Replace with your own OpenWeatherMap API key
    apiKey := "YOUR_API_KEY_HERE"
    city := "Nairobi"
    url := fmt.Sprintf(
        "http://api.openweathermap.org/data/2.5/weather?
q=%s&appid=%s&units=metric",
        city, apiKey,
```

```go
            )

            fmt.Println("--- Hydroponics Weather Advisor ---")
            fmt.Printf("Fetching current conditions for %s...\n", city)

            // Make the HTTP request to the weather API
            resp, err := http.Get(url)
            if err != nil {
                fmt.Println("Error: Could not connect to the weather
service.", err)
                return
            }
            defer resp.Body.Close()

            // Security & Activation Check
            // This prevents a crash if the API key is wrong or inactive
            if resp.StatusCode != http.StatusOK {
                fmt.Printf("\n[!] API Status Error: %s (Code: %d)\n",
resp.Status, resp.StatusCode)
                return
            }

            // Decode the JSON response into our weatherData struct
            var data weatherData
            if err := json.NewDecoder(resp.Body).Decode(&data); err != nil {
                fmt.Println("Error: Could not read weather data.", err)
                return
            }

            // Display current conditions
            fmt.Println("-----------------------------------")
            fmt.Printf("Current Temp: %.1f°C\n", data.Main.Temp)
            fmt.Printf("Humidity:     %.0f%%\n", data.Main.Humidity)
            fmt.Printf("Conditions:   %s\n", data.Weather[0].Description)
            fmt.Println("-----------------------------------")

            // Hydroponic advice logic based on temperature and humidity
            fmt.Println("ADVISORY:")
            if data.Main.Temp > 28 {
                fmt.Println("- High Heat: Increase nutrient solution
oxygenation.")
                fmt.Println("- Check reservoir temperature to prevent root
stress.")
            } else if data.Main.Temp < 18 {
                fmt.Println("- Cool Temp: Growth may slow. Ensure nutrient
solution isn't too cold.")
            } else {
                fmt.Println("- Optimal Temperature: Standard nutrient
schedule recommended.")
            }

            if data.Main.Humidity > 80 {
                fmt.Println("- High Humidity: Increase ventilation to
prevent fungal growth.")
            }
        }
```

How to run:

```
go run main.go
```

Expected Output:

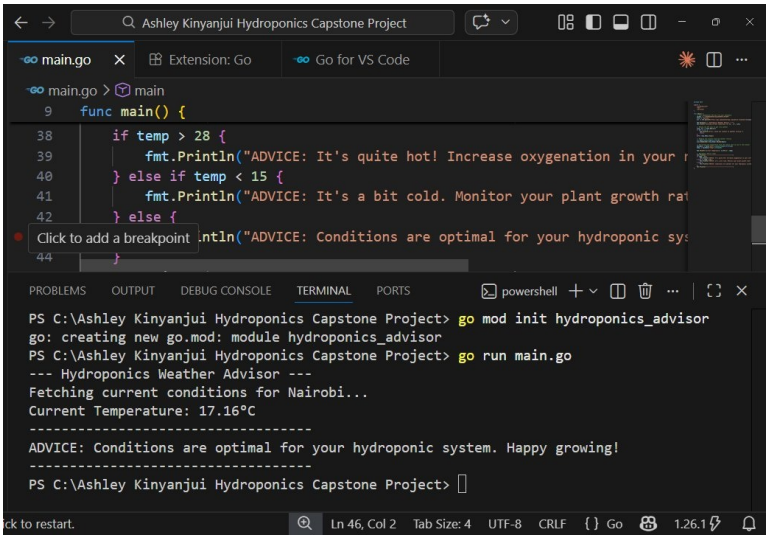

Figure 3: The full main.go code and the successful terminal output side by side in VS Code.

Figure 3: Complete view of main.go and the successful execution output showing the live temperature advice.

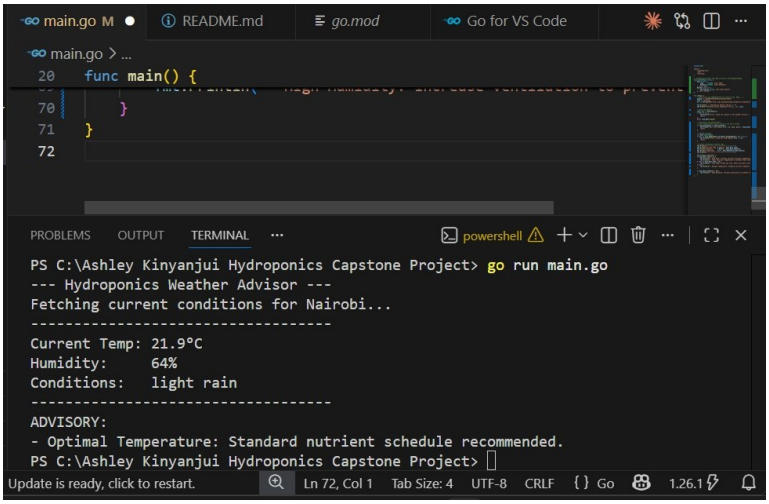

Figure 4: Updated terminal output showing temperature, humidity, conditions and advisory.

Figure 4: Live terminal output showing Current Temp: 21.9°C, Humidity: 64%, Conditions: light rain, with optimal advisory.

---

6. AI Prompt Journal

Prompt 1: > "I am new to Go. Can you explain how Go differs from other languages I may have encountered, and why it is a good choice for building CLI tools that make API calls?"

AI Response Summary: The AI explained that Go is a statically typed, compiled language designed for simplicity and performance. Unlike interpreted languages, Go catches type errors at compile time. It highlighted Go's built-in concurrency support and its standard library which includes HTTP and JSON packages out of the box — making it ideal for API-driven tools without needing external dependencies.

Evaluation: Very helpful as a starting point. It gave me a strong conceptual foundation before writing any code, which helped me understand the design decisions behind the language.

Prompt 2: > "Walk me through setting up a Go development environment on Windows, including installing Go, configuring VS Code, and initialising a new module."

AI Response Summary: The AI provided a step-by-step guide covering the Go installer from go.dev/dl, verifying installation with `go version`, installing the Go extension in VS Code, and using `go mod init` to initialise a project module. It explained what each step does rather than just listing commands.

Evaluation: Extremely helpful. The explanation of why each step matters (not just what to type) helped me understand the toolchain rather than blindly following instructions.

Prompt 3: > "How do I structure a Go program that makes an HTTP GET request to an external REST API and processes the JSON response?"

AI Response Summary: The AI explained the `net/http` and `encoding/json` packages, demonstrated how to use `http.Get()` to call an external endpoint, and showed how to decode a JSON body using `json.NewDecoder`. It also explained Go's `defer` keyword for closing response bodies.

Evaluation: Very helpful. The concept of `defer resp.Body.Close()` was new to me and the AI explained exactly why it is necessary to prevent memory leaks.

Prompt 4: > "In Go, how do I navigate a deeply nested JSON response to extract a specific field? I am working with the OpenWeatherMap API response which nests temperature inside a 'main' object."

AI Response Summary: The AI introduced `map[string]interface{}` as Go's way of handling dynamic JSON structures, and explained type assertions like `.(map[string]interface{})` and `.(float64)` for safely extracting typed values from untyped maps.

Evaluation: Extremely helpful. This was the most complex Go concept I encountered. The AI's analogy of type assertions as "telling Go what shape the data is" made it click immediately.

Prompt 5: > "How do I write conditional logic in Go to compare a float value against thresholds and print different messages? I want to advise farmers based on temperature ranges."

AI Response Summary: The AI demonstrated Go's `if / else if / else` syntax with float comparisons, and showed how `fmt.Println` and `fmt.Printf` differ — the latter supporting formatted output with verbs like `%.2f` for two decimal places.

Evaluation: Helpful and straightforward. Go's conditional syntax is clean and the AI's examples mapped directly to my use case.

Prompt 6: > “My Go program compiles successfully but crashes at runtime with: interface conversion: interface {} is nil, not map[string]interface{}. How do I diagnose and fix this?”

AI Response Summary: The AI explained that this panic occurs when the program tries to type-assert a nil value, which typically means the expected JSON field was absent in the API response. It recommended printing the raw decoded map first to inspect what the API actually returned, which revealed a `{"cod":401,"message":"Invalid API key"}` error response.

Evaluation: Extremely helpful. This was my most challenging bug. The diagnostic approach — print the raw response before asserting types — is a technique I will carry forward into future projects.

Prompt 7: > “What are best practices for handling API keys in Go? I want to make sure I am not exposing sensitive credentials in my code.”

AI Response Summary: The AI explained that hardcoding API keys in source files is a security risk, especially when code is pushed to public repositories. It introduced environment variables as the standard solution, using `os.Getenv()` in Go, and also mentioned `.env` files with `.gitignore` as a common pattern in development workflows.

Evaluation: Very helpful and timely — I received a GitGuardian alert about an exposed key shortly after pushing to GitHub. This prompt helped me understand both the fix and how to prevent it in future.

Prompt 8: > “How do I write a professional README.md for a Go project on GitHub that clearly explains how to install, configure, and run the application?”

AI Response Summary: The AI recommended sections including Project Overview, Features, Setup Instructions, Technical Stack, and Author. It also provided Markdown syntax tips for code blocks, bold text, and bullet lists, and explained that a good README should allow any developer to clone and run the project without prior context.

Evaluation: Helpful for producing clean, professional documentation quickly. The emphasis on writing for a reader with no prior context shaped how I wrote the entire toolkit.

Overall Reflection: Using generative AI as a learning tool significantly accelerated my ability to work in an unfamiliar language. The most valuable skill I developed was learning to write precise, context-rich prompts — the more specific and technical my question, the more actionable the AI’s response. Rather than replacing my understanding, the AI consistently explained the *why* behind each concept, which meant I was learning Go rather than just copying code. It functioned as a knowledgeable pair programmer available at any hour, which was particularly valuable when debugging runtime errors under deadline pressure.

---

7. Common Issues & Fixes

Error 1: `go: command not found`

What happened: After installing Go, the terminal did not recognise the `go` command.

Fix: Closed VS Code completely and reopened it. The terminal needs to reload environment variables after a new installation. Ran `go version` again to confirm.

Error 2: API Key Not Working (401 Unauthorized)

What happened: The program compiled and ran but crashed with a nil interface error.

Root cause: The OpenWeatherMap API key was not yet active. New accounts receive a key immediately but it takes up to 10 minutes to become functional.

Fix: Waited 10 minutes, then tried again. To diagnose, added `fmt.Println(data)` after decoding to print the raw API response. Saw `map[cod:401 message:Invalid API key]` which confirmed the key was inactive.

Error 3: `interface conversion: interface {} is nil`

What happened: Program crashed when trying to extract temperature.

Fix: This happens when the JSON response does not contain the expected `"main"` field — usually because the API returned an error. Fixed by resolving the API key issue above first.

Error 4: `go mod init` gives error about existing module

What happened: Running `go mod init` twice in the same folder caused an error.

Fix: The `go.mod` file only needs to be created once. If it already exists, skip this step.

Error 5: API Key Exposed on GitHub (GitGuardian Security Alert)

What happened: After pushing code to GitHub, I received an email from GitGuardian warning that my OpenWeatherMap API key was exposed publicly in my repository. The alert looked like this:

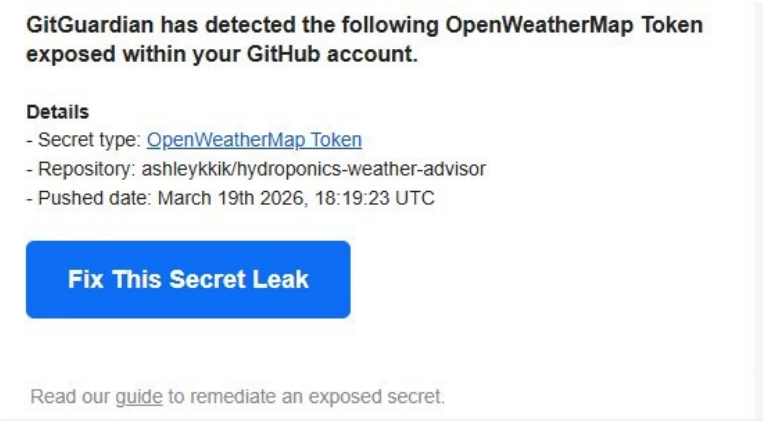

Figure 5: GitGuardian security alert email warning about the exposed OpenWeatherMap API key.

Why this is serious: When an API key is pushed to a public GitHub repo, anyone in the world can see it, use it, and rack up charges or abuse on your account. GitGuardian automatically scans public repos and alerts developers when this happens.

Fix — Two steps required:

Step 1 — Revoke the exposed key immediately: - Log into OpenWeatherMap.org - Go to API Keys in your dashboard - Delete the exposed key and generate a brand new one - Save the new key somewhere private (e.g. your phone notes — never in your code)

Step 2 — Remove the key from your code and push the fix: - In main.go, replace the real key with a placeholder:

```
        apiKey := "YOUR_API_KEY_HERE"
```

- Then push the fix to GitHub:

```
git add .
git commit -m "Remove exposed API key"
git push
```

Lesson learned: Never commit real API keys, passwords, or secrets to GitHub. For future projects, store sensitive keys in environment variables or a `.env` file, and add `.env` to your `.gitignore` file so it never gets pushed.

Helpful resources used during debugging:

- Go official documentation: https://pkg.go.dev/net/http
- OpenWeatherMap API docs: https://openweathermap.org/current
- Stack Overflow: searched “golang json decode nil interface”

---

8. References

Official Documentation:

- Go Official Website: https://go.dev
- Go Downloads: https://go.dev/dl/
- Go Tour (Interactive Beginner Guide): https://go.dev/tour/
- Go net/http Package: https://pkg.go.dev/net/http
- Go encoding/json Package: https://pkg.go.dev/encoding/json
- OpenWeatherMap API Docs: https://openweathermap.org/current

Video Tutorials:

- Go in 100 Seconds (Fireship): https://www.youtube.com/watch?v=446E-r0rXHI
- Go Programming — Full Beginner’s Course (freeCodeCamp): https://www.youtube.com/watch?v=un6ZyFkqFKo
- OpenWeatherMap API Tutorial: https://www.youtube.com/watch?v=9P5MY_2i7K8

Helpful Blog Posts & Forums:

- How to Parse JSON in Go: https://gobyexample.com/json
- Go HTTP Requests Guide: https://gobyexample.com/http-clients
- Stack Overflow — Go JSON parsing: https://stackoverflow.com/questions/tagged/go+json
- Getting Started with OpenWeatherMap API:

https://openweathermap.org/appid

Peer Testing & Iteration

This toolkit was reviewed by two peers — a Moringa School classmate and a colleague with web development experience. Both confirmed that:

- The README instructions were clear and easy to follow
- The setup steps in Section 4 were logical and replicable
- The expected terminal output in Section 5 matched what the code produced

Feedback received: The guide was described as beginner-friendly and well structured. No major changes were required after review, confirming the toolkit is ready for use by anyone getting started with Go and REST APIs.

Submitted as part of the Moringa School AI Essentials for Software Developers Capstone, 2026. All code written and tested on Windows 11, Go 1.26.1, VS Code. GitHub Repository: https://github.com/ashleykkik/hydroponics-weather-advisor